



НИУ ВШЭ - Нижний Новгород

January 10, 2025

Алгоритмы и структуры данных

Лекция 6. Структуры данных

Илья Сергеевич Бычков

ibychkov@hse.ru



Лекция 6. Структуры данных



- 0. План лекции
- 1. **Стек с поддержкой минимума/максимума**
- 2. Хэш-таблица (hash table)



Задача: Добавить возможность нахождения минимума в стеке за $O(1)$, сохранив такой же асимптотику добавления и удаления элементов из стека.

Решение: Хранить в стеке не сами элементы, а пары: элемент и минимум в стеке, начиная с этого элемента и ниже.



- 0. План лекции
- 1. Стек с поддержкой минимума/максимума
- 2. Хэш-таблица (hash table)



Ассоциативный массив

Ключи и значения

Задача

Хранить данные и уникальные идентификаторы, по которым эти данные можно различать. Идентификатор будем называть **ключ**, а данные - **значение**. Таким образом, необходимо уметь хранить пары (ключ, значение) и работать с ними.

Абстракция ключ-значение

- Вставка значения "по ключу"
- Дан ключ, необходимо найти значение



Direct-address tables

Простое решение в стиле сортировки подсчётом (counting sort).

Ключи - целые числа в некотором интервале $[0, n - 1]$.

Значения - произвольный объект.

Абстракция ключ-значение

- Вставка значения "по ключу"
- Дан ключ, необходимо найти значение

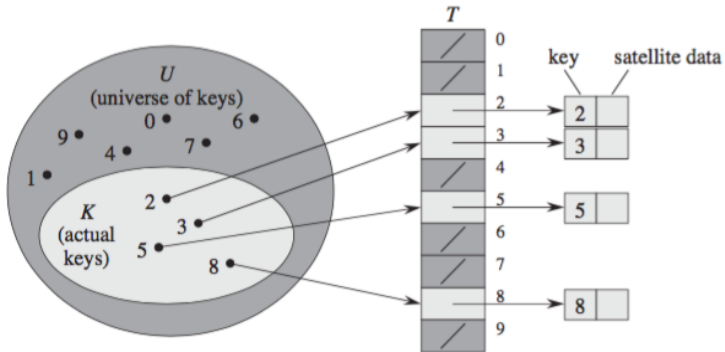
Операции поиска, вставки и удаления работают за $O(1)$, так как они реализованы в виде работы с конкретными элементами массива.



Ассоциативный массив

Direct-address tables

Direct-address tables



(1) Источник - Stanford (Jessica Su) - Cormen et al.



Direct-address tables

Операции поиска, вставки и удаления работают за $O(1)$, так как они реализованы в виде работы с конкретными элементами массива.

DIRECT-ADDRESS-SEARCH(T, k)

1 **return** $T[k]$

DIRECT-ADDRESS-INSERT(T, x)

1 $T[x.key] = x$

DIRECT-ADDRESS-DELETE(T, x)

1 $T[x.key] = \text{NIL}$

(2) Источник - Stanford (Jessica Su) - Cormen et al.



Ассоциативный массив

Ассоциативный массив

Ассоциативный массив/словарь/отображение (dictionary/map)

это структура данных, обобщающая понятие массива, позволяющая хранить и обрабатывать данные по произвольным **ключам**.

Обобщение массива Индексы массива не обязательно могут быть числами от 0 до N-1

Поддержка в современных языках

- Сторонние библиотеки: C, VisualBasic, Standard ML, bash, ...
- Встроенные библиотеки: Java, C#, C++, Scala, ...
- Поддержка на уровне языка: Awk, Perl, PHP, Tcl, JavaScript, Python, Ruby, Lua



Пример - DNS lookup

- Добавить URL с настоящим адресом
- Дан URL, необходимо узнать адрес

domain name	IP address
www.cs.princeton.edu	128.112.136.11
www.princeton.edu	128.112.128.15
www.yale.edu	130.132.143.21

(3) Источник - [Sedjwick, Wayne - algs4.cs.princeton.edu](http://Sedjwick.Wayne-algs4.cs.princeton.edu)



Ассоциативный массив

Ассоциативный массив

Ассоциативный массив. Каждый элемент данных (значение) ассоциируется с некоторым идентификатором (ключ)

Функциональность:

- `put(key, value)`, обычно `a[key] = value`
- `put(key, value)` перезаписывает старое значение новым
- `get(key)`, обычно `a[key]`
- `get(key)` возвращает `null` если не существует значения по заданному ключу

Легкая реализация `contains(key)` и `delete(key)`



Структура данных. Связный список пар ключ-значение

Поиск. Исследуем все элементы пока не найдем совпадение или не закончится список

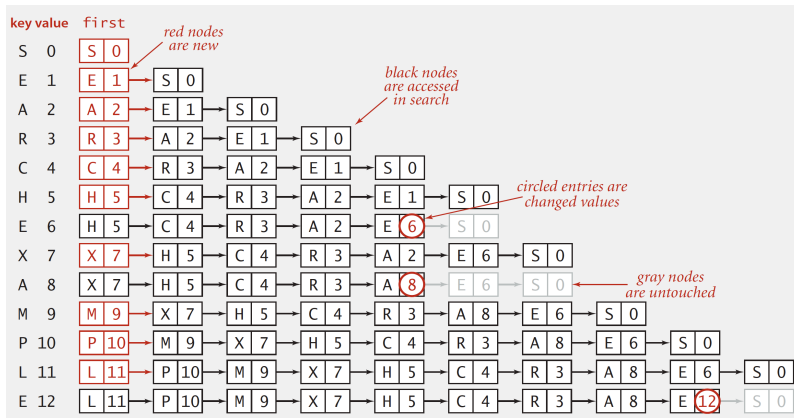
Вставка. Исследуем все элементы пока не найдем совпадения, если не нашли – вставляем в начало



Ассоциативный массив

Возможные реализации

Структура данных. Связный список пар ключ-значение



(4) Источник - [Sedjwick, Wayne - algs4.cs.princeton.edu](http://algs4.cs.princeton.edu)

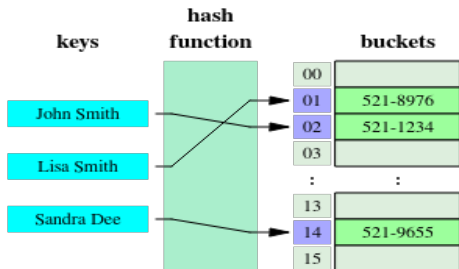


Ассоциативный массив

Возможные реализации

Структура данных. Сохраняем элементы в виде таблицы(массива) при помощи хэш-функции.

Хэш-функция. Функция для вычисления индекса массива по ключу.



(5) Источник - algs4.cs.princeton.edu



Ассоциативный массив

Хэш-функция

Хэш-функция. Функция для вычисления индекса массива по ключу.

Проблемы.

- Вычисление хэш-функции
- Тест на равенство: Проверить что два ключа равны
- Разрешение коллизий: Обработка ситуации когда два ключа имеют одинаковое значение хэш-функции

Classic space-time tradeoff

- Нет ограничений на память: тривиальная хэш-функция, где значение ключа и есть индекс
- Нет ограничений на время: тривиальный алгоритм разрешения коллизий – использование линейного поиска
- Ограничения: hashing (the real world)



Хэш-функция

Идеальный случай: ключи равномерно распределены по массиву

Эффективность по вычислениям

Пример. Телефонные номера

Плохой вариант: первые 3 цифры

Хороший вариант: последние 3 цифры

Нетривиальная задача на практике.

Необходим различный подход для разных типов ключей



Ассоциативный массив

Хэш-функция

Хэш-функция. Функция для вычисления индекса массива по ключу.

Java library implementations

```
public final class Integer
{
    private final int value;
    ...

    public int hashCode()
    { return value; }
}
```

```
public final class Boolean
{
    private final boolean value;
    ...

    public int hashCode()
    {
        if (value) return 1231;
        else      return 1237;
    }
}
```

```
public final class Double
{
    private final double value;
    ...

    public int hashCode()
    {
        long bits = doubleToLongBits(value);
        return (int) (bits ^ (bits >> 32));
    }
}
```

convert to IEEE 64-bit representation;
xor most significant 32-bits
with least significant 32-bits

Warning: -0.0 and +0.0 have different hash codes

(6) Источник - Sedjwick, Wayne - algs4.cs.princeton.edu



Хэширование строк

- $A=a[0]a[1]a[2]...a[i]$ – символы
- Использовать все символы – хорошая практика, иначе возможно большое количество коллизий
- Например, если $a[0]$ не используется, $\text{hash}("aa") = \text{hash}("ba") = \dots = \text{hash}("za")$

Полиномиальный хэш

$$h(a_1 a_2 \cdots a_n) = (a_1 x^{n-1} + a_2 x^{n-2} + a_3 x^{n-3} + \cdots + a_n x^0) \bmod q$$

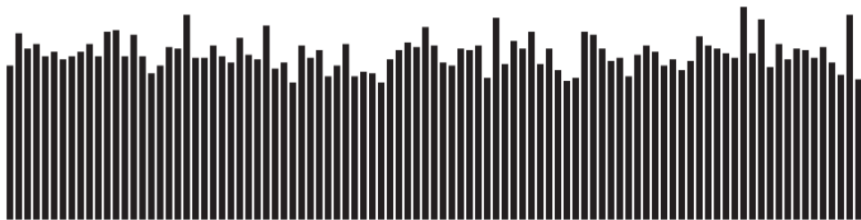
(7) Источник - Добавить



Ассоциативный массив

Хэш-функция

Хэш-функция. Функция для вычисления индекса массива по ключу.



Hash value frequencies for words in Tale of Two Cities ($M = 97$)

Java's String data uniformly distribute the keys of Tale of Two Cities

(8) Источник - [Sedjwick, Wayne - algs4.cs.princeton.edu](http://algs4.cs.princeton.edu)

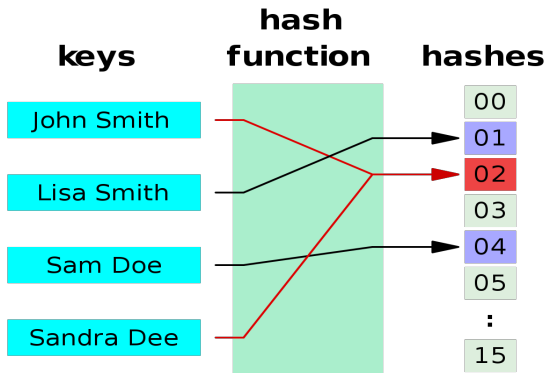


Ассоциативный массив

Коллизия

Коллизия. Два различных ключа имеют одинаковое хэш-значение.

Задача. Придумать механизм для эффективного разрешения коллизий.





Метод цепочек (chain hashing). Используем массив из $M < N$ связанных списков. [H. P. Luhn, IBM 1953] .

Хэширование. Отображение в целое число между 0 и $M - 1$.

Вставка. вставка в начало цепи i (если элемента нет).

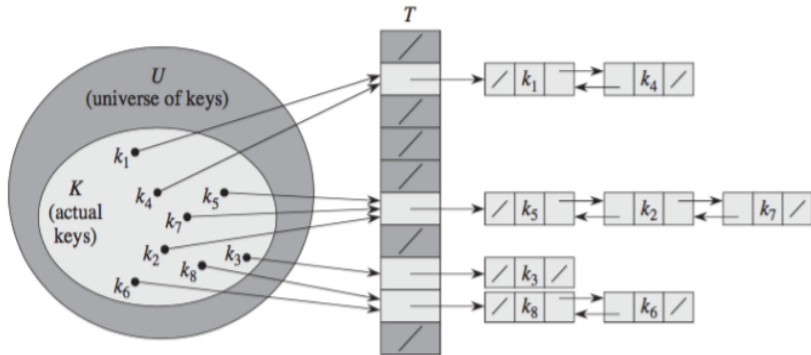
Поиск. необходимость поиска только в цепи i .



Ассоциативный массив

Метод цепочек

Метод цепочек (chain hashing). Используем массив из $M < N$ связанных списков. [Н. Р. Luhn, IBM 1953].



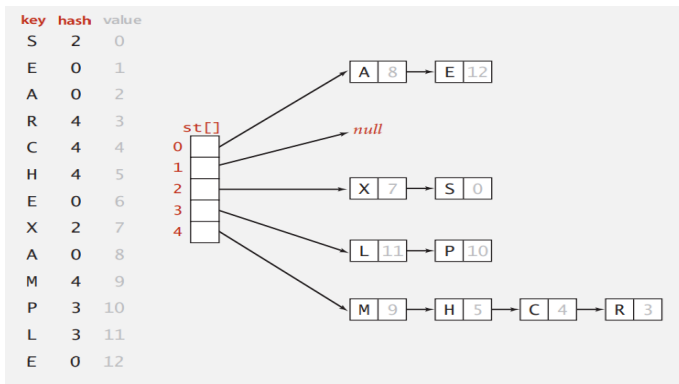
(10) Источник - Stanford (Jessica Su) - Cormen et al.



Ассоциативный массив

Метод цепочек

Метод цепочек (chain hashing). Используем массив из $M < N$ связанных списков. [Н. P. Luhn, IBM 1953].



(11) Источник - Sedjwick, Wayne - algs4.cs.princeton.edu



Издержки. Число исследований (**probes**) для поиска и вставки пропорционально N / M .

- Слишком большое M — слишком много пустых цепочек
- Слишком маленькое M — слишком длинные цепочки
- Практический результат: $M \approx N / 4$ — константное время операций

Цель. Средняя длина цепи N / M — константа.

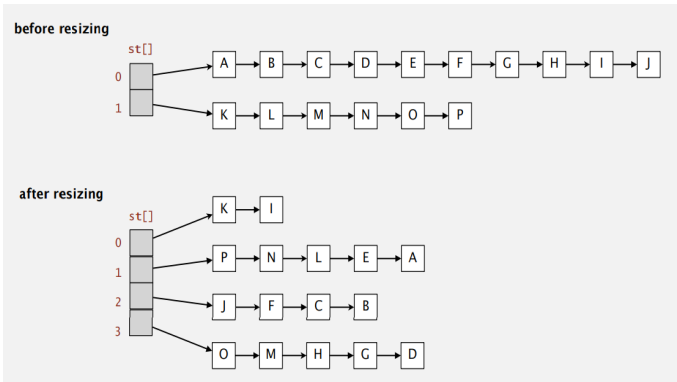
- Удваиваем значение M когда $N / M \geq 8$
- Делим пополам значение M когда $N / M \leq 2$
- Повторное хэширование после изменения размеров



Хэш-таблица

Метод цепочек

Метод цепочек (chain hashing). Используем массив из $M < N$ связанных списков. [Н. Р. Luhn, IBM 1953].



(12) Источник - Sedjwick, Wayne - algs4.cs.princeton.edu

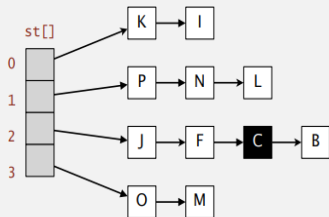


Хэш-таблица

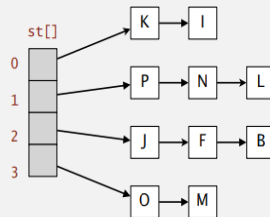
Метод цепочек

Удаление элемента

before deleting C



after deleting C



(13) Источник - Sedjwick, Wayne - algs4.cs.princeton.edu



Хэш-таблица

Метод открытой адресации

Метод открытой адресации (open addressing). При возникновении коллизии ищем первый доступный пустой слот и помещаем туда ключ/значение [Amdahl-Boehme-Rocherster-Samuel, IBM 1953].

st[0]	jocularly
st[1]	<i>null</i>
st[2]	listen
st[3]	suburban
⋮	<i>null</i>
st[30000]	browsing

(14) Источник - Sedjwick, Wayne - algs4.cs.princeton.edu



Хэширование. Отображение ключей на интервал $[0, M-1]$

Вставка. Если индекс в массиве свободен – записываем значение туда. Иначе пробуем $i+1$, $i+2$ и тд

Note. Размер массива должен вместить все пары ключ-значение.

search K

$$\text{hash}(K) = 5$$

(15) Источник - [Sedjwick, Wayne - algs4.cs.princeton.edu](https://algs4.cs.princeton.edu)



Хэш-таблица

Метод открытой адресации

```
public class LinearProbingHashST<Key, Value>
{
    private int M = 30001;
    private Value[] vals = (Value[]) new Object[M];
    private Key[] keys = (Key[]) new Object[M];

    private int hash(Key key) { /* as before */ }

    private Value get(Key key) { /* previous slide */ }

    public void put(Key key, Value val)
    {
        int i;
        for (i = hash(key); keys[i] != null; i = (i+1) % M)
            if (keys[i].equals(key))
                break;
        keys[i] = key;
        vals[i] = val;
    }
}
```

(16) Источник - [Sedjwick, Wayne - algs4.cs.princeton.edu](https://algs4.cs.princeton.edu)



Linear probing: Let $h(k, i) = (h'(k) + i) \bmod m$ for $i = 0, 1, \dots, m - 1$. In other words, first we examine the slot $T[h'(k)]$, i.e. the slot given by the auxiliary hash function. Then we probe the slot next to it, and then the slot next to that slot, etc, wrapping around to the beginning of the array if necessary.

(17) Источник - [Stanford \(Jessica Su\)](#) - Cormen et al.



Quadratic probing: Let $h(k, i) = (h'(k) + c_1i + c_2i^2) \bmod m$. This is a better method than linear probing, but we may have to select the constants carefully. Also, if two keys have the same initial probe position, then their probe sequences are the same, which means we still get clustering, but to a lesser extent.

(18) Источник - Stanford (Jessica Su) - Cormen et al.



Double hashing: Here we use two hash functions, h_1 and h_2 , and let $h(k, i) = (h_1(k) + ih_2(k)) \bmod m$. This is a great method and the permutations produced have many of the characteristics of randomly chosen permutations.

(19) Источник - [Stanford \(Jessica Su\)](#) - Cormen et al.



Many improved versions have been studied.

Two-probe hashing [separate-chaining variant]

Hash to two positions, insert key in shorter of the two chains.

Double hashing [linear-probing variant]

Use linear probing, but skip a variable amount, not just 1 each time.

Effectively eliminates clustering.

Cuckoo hashing [linear-probing variant]

Hash key to two positions; insert key into either position; if occupied, reinsert displaced key into its alternative position.